VEER NARMAD SOUTH GUJARAT UNIVERSITY, SURAT

VIMAL TORMAL PODDAR BCA COLLEGE

TYBCA (SEMESTER-6)

2026-27

PROJECT REPORT ON

Trade Tracker

GUIDED BY:

MS KRUPA MA'AM

SUBMITTED BY:

GANESH BHOGARE
NIDHESH DUBEDI
HITESH PATIL

Abstract

The modern financial trading environment requires accurate record-keeping, performance monitoring, and effective risk management.
Traditional methods of maintaining trading records manually often lead to data inconsistencies, calculation errors, and inefficient performance analysis.

This project report presents the design and development of **"Trade Tracker,"** a webbased trading journal system developed using the **MERN stack (MongoDB, Express.js, React.js, and Node.js)**.
The system provides a centralized platform for traders to record, manage, and analyze their trading activities in real time.

The application includes secure user authentication, trade management features, performance analytics dashboards, and risk monitoring tools.
The system architecture follows a modular and scalable design, ensuring maintainability and future enhancements.

The frontend interface is built using modern web technologies to provide a responsive and user-friendly experience, while the backend ensures secure data handling and efficient database operations.

The Trade Tracker system improves trading discipline, enhances performance evaluation, and supports better decision-making for traders.

Acknowledgement

We would like to express our sincere gratitude to our project guide, **Ms**

Krupa Patel,

for her valuable guidance, encouragement, and continuous support throughout the development of this project.
Her expertise and suggestions helped us to understand complex concepts and successfully complete this project.

We would also like to thank the faculty members of **Vimal**

Tormal Poddar BCA College

for providing the necessary facilities and academic environment required for completing this project.

Finally, we would like to thank our parents, friends, and well-wishers for their constant support and motivation during the project development period.

GANESH BHOGARE
NIDHESH DUBEDI

HITESH PATIL

Contents

Certificate i
Abstract ii
Acknowledgement iii
Contents iv
List of Figures vi

---

1 Introduction 1

1.1 Project Background and Motivation 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Project Rationale 2
1.5 Scope 2
1.6 Methodology ..... 3 **1.7 Report Organization** 3

---

2 Literature Survey & Technology Stack 5

2.1 Literature Survey 5
2.1.1 Analysis of Existing Trading Platforms 5
2.1.2 Academic Research 5
2.2 Existing System Limitations 6
2.3 Technology Rationale and Stack 6

2.3.1 Hardware Requirements (Development) 6
2.3.2 Software Configuration (Development) 6
2.3.3 Technology Justification 7

---

3 System Analysis and Requirements 8

3.1 Feasibility Study 8
3.2 Requirements Specification 8

3.2.1 Functional Requirements 8
3.2.2 Non-Functional Requirements 10

4 System Design and Architecture 11

4.1 System Architecture 11
4.2 Data Flow Diagrams (DFDs) 13

4.2.1 Context-Level Diagram (DFD-0) 13
4.2.2 Level-1 Diagram (DFD-1) 15
4.2.3 Level-2 Diagram (Trade Management) 16

4.3 Database Design 17

4.3.1 Database Schema (E-R Diagram) 17
4.3.2 Data Dictionary 18

5 UML Modeling 20

5.1 Use Case Diagram 20
5.2 Activity Diagrams 22

5.2.1 Activity Diagram for User Registration 22
5.2.2 Activity Diagram for Trade Entry 23

5.3 Sequence Diagrams 24

5.3.1 Sequence Diagram for User Login ..... 24 **5.3.2 Sequence Diagram for Trade Recording** 25

6 Implementation and UI Design 26

6.1 Implementation Details 26
6.2 Module Breakdown ..... 26 **6.3 UI/UX Design (Input/Output Screens)** 27

7 System Testing 37

7.1 Testing Strategy ..... 37 **7.2 Sample Test Cases** 37

8 Conclusion 38

8.1 Project Summary 38

8.2 Limitations 38
8.3 Future Scope 39

List of Figures

Figure 4.1 Three-Tier Architecture Diagram 11
Figure 4.2 Context-Level Diagram (DFD-0) 13
Figure 4.3 Level-1 Diagram (DFD-1) 15
Figure 4.4 Level-2 Diagram (Add Trade Process) 16
Figure 4.5 Entity-Relationship (E-R) Diagram 18
Figure 5.1 Use Case Diagram 20
Figure 5.2 Activity Diagram for User Registration 22
Figure 5.3 Activity Diagram for Add Trade 23
Figure 5.4 Sequence Diagram for User Login 24
Figure 6.1 UI Screenshot: Dashboard 29
Figure 6.2 UI Screenshot: Login Page 30
Figure 6.3 UI Screenshot: Add Trade Page 31
Figure 6.4 UI Screenshot: Trade History Page 32

Chapter 1 Introduction

1.1 Project Background and Motivation

The rapid growth of online financial markets has increased the demand for efficient tools to manage trading activities.
Traders require accurate records, performance analysis, and proper risk management to make informed decisions.
However, many traders still rely on manual methods such as spreadsheets or handwritten notes to track their trades, which can lead to errors and data inconsistencies.

The motivation behind the development of the **Trade Tracker** system is to provide a reliable and user-friendly web application that helps traders maintain organized trading records.
The system allows users to record trade details, monitor profit and loss, analyze performance, and manage risk effectively.

By digitizing the trading process, the Trade Tracker system improves efficiency, reduces manual errors, and enhances decision-making capabilities for traders.
The application also provides a centralized platform for storing trading data securely and accessing it anytime through an internet connection.

1.2 Problem Statement

In traditional trading management systems, traders often face several challenges that affect their performance and record management.

The common problems include:

- Manual recording of trading transactions, which increases the chances of calculation errors
- Difficulty in tracking profit and loss over time
- Lack of proper performance analysis tools
- Inefficient data storage and retrieval methods

• Limited risk management and reporting features

These issues make it difficult for traders to evaluate their trading performance and improve their strategies.

The Trade Tracker system addresses these challenges by providing an automated platform for recording trades, analyzing performance, and managing trading data efficiently.

1.3 Objectives

The main objectives of the Trade Tracker system are as follows:

1. To design and develop a web-based trading journal system using the MERN stack.
2. To provide a secure login and user authentication system.
3. To allow users to record, update, and delete trading transactions.
4. To generate reports and analytics for evaluating trading performance.
5. To improve risk management and decision-making capabilities.
6. To provide a user-friendly and responsive interface.

1.4 Project Rationale

The Trade Tracker system is developed to provide an efficient solution for managing trading activities in a structured and automated manner.

The system introduces the following advantages:

- Centralized Data Management: All trading records are stored in a single database for easy access and analysis.
- Performance Monitoring: Users can track profit, loss, and trading statistics in real time.
- Secure Authentication: The system ensures that only authorized users can access their trading data.
- Improved Decision-Making: Analytical reports help traders evaluate their strategies and performance.
- Scalable Architecture: The system is designed to support future enhancements and additional features.

1.5 Scope

The scope of the Trade Tracker application includes the following functionalities:

- User Registration and Login System
- Recording and Managing Trade Details
- Viewing Trade History and Performance Reports
- Risk Management and Profit/Loss Analysis
- Secure Data Storage and Retrieval
- Responsive Web Interface for Users

The system is designed primarily for individual traders who want to monitor their trading performance and maintain accurate trading records.

1.6 Methodology

The development of the Trade Tracker system follows the **Software Development Life Cycle (SDLC)** methodology to ensure systematic and structured development.

The development process consists of the following stages:

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

Each phase ensures that the system is developed efficiently and meets user requirements.

1.7 Report Organization

This report is organized into eight chapters to provide a detailed explanation of the Trade Tracker system.

- **Chapter 1 (Introduction):** Describes the background, problem statement, objectives, scope, and methodology of the project.
- **Chapter 2 (Literature Survey & Technology Stack):** Discusses existing systems and explains the technologies used in the project.
- **Chapter 3 (System Analysis and Requirements):** Defines system requirements and feasibility analysis.
- **Chapter 4 (System Design and Architecture):** Presents system architecture, data flow diagrams, and database design.
- **Chapter 5 (UML Modeling):** Describes system behavior using UML diagrams. • **Chapter 6 (Implementation and UI Design):** Explains system modules and user interface design.
- **Chapter 7 (System Testing):** Describes testing methods and test cases. • **Chapter 8 (Conclusion):** Summarizes the project and suggests future improvements.

Chapter 2 Literature Survey & Technology Stack

2.1 Literature Survey

To understand the current landscape of trading management systems, a survey of existing trading platforms and relevant research was conducted. This study helped identify the strengths and limitations of existing solutions and guided the development of the Trade Tracker system.

2.1.1 Analysis of Existing Trading Platforms

Several online trading and trading journal platforms are currently available in the market.
These platforms provide various features such as trade recording, analytics, and performance tracking.
However, they also have certain limitations that affect usability and accessibility.

- **MetaTrader (MT4/MT5):**

These platforms are widely used for trading in financial markets.
They provide advanced charting tools and automated trading capabilities.
However, they do not offer comprehensive trade journaling and performance tracking features for personal analysis.

- **TradingView:**

TradingView is a popular platform that provides real-time charts and technical analysis tools.
It allows traders to analyze market trends and share trading ideas.
However, it requires a subscription for advanced features and does not focus primarily on trade record management.

- **Excel-Based Trading Journals:**

Many traders use spreadsheets to record trading data.
While spreadsheets are flexible and easy to use, they lack automation, data security, and advanced analytics features.

This analysis shows that there is a need for a dedicated trading journal system that provides secure data storage, automated analysis, and user-friendly functionality.

2.1.2 Academic Research

Research studies in financial technology highlight the importance of maintaining accurate trading records and analyzing trading performance. Effective trade management systems help traders identify strengths and weaknesses in their strategies.

Studies also emphasize the role of technology in improving decision-making through data visualization and performance metrics.
These findings influenced the design of the Trade Tracker system, which focuses on performance monitoring, risk management, and data security.

2.2 Existing System Limitations

The existing trading management systems have several limitations that affect their efficiency and usability.

The major limitations include:

- Dependence on manual record-keeping methods
- Lack of centralized data storage
- Limited performance analysis tools
- Difficulty in tracking trading performance over time
- Inadequate security and data protection measures

These limitations highlight the need for an improved system that provides automation, security, and efficient data management.

The Trade Tracker system is designed to overcome these challenges by offering a centralized and automated trading journal platform.

2.3 Technology Rationale and Stack

The selection of technologies for the Trade Tracker system was based on performance, scalability, development efficiency, and community support.

The system is developed using the **MERN stack**, which includes MongoDB, Express.js, React.js, and Node.js.
These technologies provide a reliable and scalable environment for building modern web applications.

2.3.1 Hardware Requirements (Development)

Table 2.1: Minimum Hardware Requirements

Hardware Component — Specification

Processor — Intel Core i5 or AMD Ryzen 5 (or equivalent)
RAM — Minimum 8 GB (16 GB Recommended)
Storage — 20 GB free disk space
Internet — Stable broadband connection
Display — 1920 × 1080 resolution

2.3.2 Software Configuration (Development)

Table 2.2: Software Configuration

Component — Specification

Operating System — Windows 10 / Windows 11 / Linux
Development IDE — Visual Studio Code
Programming Language — JavaScript (ES6+)
Frontend Framework — React.js
Backend Framework — Node.js and Express.js
Database — MongoDB
Version Control — Git and GitHub
Web Browser — Google Chrome / Microsoft Edge

2.3.3 Technology Justification

- **React.js (Frontend):**

React.js is used to build the user interface of the Trade Tracker system.
It allows developers to create reusable components and provides fast rendering performance.

- **Node.js (Backend):**

Node.js is used to handle server-side operations and manage user requests. It provides high performance and supports asynchronous processing.

- **Express.js (Framework):**

Express.js is used to create RESTful APIs and manage application routing. It simplifies backend development and improves system efficiency.

- **MongoDB (Database):**

MongoDB is a NoSQL database used to store trading data in a flexible and scalable format.
It supports fast data retrieval and easy integration with Node.js applications.

- **Git and GitHub (Version Control):**

Git and GitHub are used to manage source code and track changes during development.
They help developers collaborate efficiently and maintain project versions.

Chapter 3

System Analysis and Requirements

3.1 Feasibility Study

A feasibility study was conducted to determine the practicality and viability of the Trade Tracker system.
This study evaluates whether the system can be successfully developed and implemented using available resources and technology.

The feasibility of the project is analyzed under the following categories:

Technical Feasibility
The Trade Tracker system is developed using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js.
These technologies are widely used in modern web development and have strong community support.
The system can be implemented using standard development tools such as Visual Studio Code and web browsers.
Therefore, the technical risk associated with the project is low.

Operational Feasibility
The system is designed with a user-friendly interface that allows traders to record and manage their trades easily.
Users require minimal training to operate the system, and the interface provides clear navigation and functionality.
The system improves operational efficiency by automating trading record management and performance analysis.

Economic Feasibility
The Trade Tracker system is cost-effective because it uses open-source technologies.
There is no requirement for expensive software licenses or hardware upgrades.
Deployment can be performed using free or low-cost hosting services, making the system financially feasible.

Legal Feasibility
The system ensures data privacy and security by implementing secure authentication and encrypted password storage.

All user data is handled responsibly and in compliance with data protection regulations.
There are no legal restrictions preventing the development or use of this system.

The feasibility study concludes that the Trade Tracker system is technically, operationally, economically, and legally feasible.

3.2 Requirements Specification

This section defines the specific requirements that the Trade Tracker system must fulfill to operate effectively.
System requirements are divided into functional and non-functional requirements.

3.2.1 Functional Requirements

Functional requirements describe the specific functions and operations that the system must perform.

User Role

- FR-U-01: The system shall allow users to register and log in securely.
- FR-U-02: The system shall allow users to add new trading records.
- FR-U-03: The system shall allow users to view and manage their trading history.
- FR-U-04: The system shall provide performance analysis and reporting features.
- FR-U-05: The system shall allow users to update or delete existing trade records.
- FR-U-06: The system shall allow users to view profit and loss summaries.
- FR-U-07: The system shall store trading data securely in the database.
- FR-U-08: The system shall allow users to log out securely from the system.

Admin Role

- FR-A-01: The system shall allow the administrator to log in securely.
- FR-A-02: The system shall allow the administrator to manage user accounts.
- FR-A-03: The system shall allow the administrator to monitor system activity.

- FR-A-04: The system shall allow the administrator to manage trading data.
- FR-A-05: The system shall allow the administrator to generate system reports.

3.2.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes and performance standards of the system.

- **NFR-01 Security:**
The system must ensure secure user authentication and protect sensitive data using encryption techniques.

- **NFR-02 Performance:**
The system must provide fast response time and efficient data processing.

- **NFR-03 Usability:**
The system interface must be simple, intuitive, and easy to use.

- **NFR-04 Reliability:**
The system must operate reliably without frequent failures or errors.

- **NFR-05 Scalability:**
The system must support future expansion and additional features.

- **NFR-06 Maintainability:**
The system must be easy to maintain and update when necessary.

Chapter 4

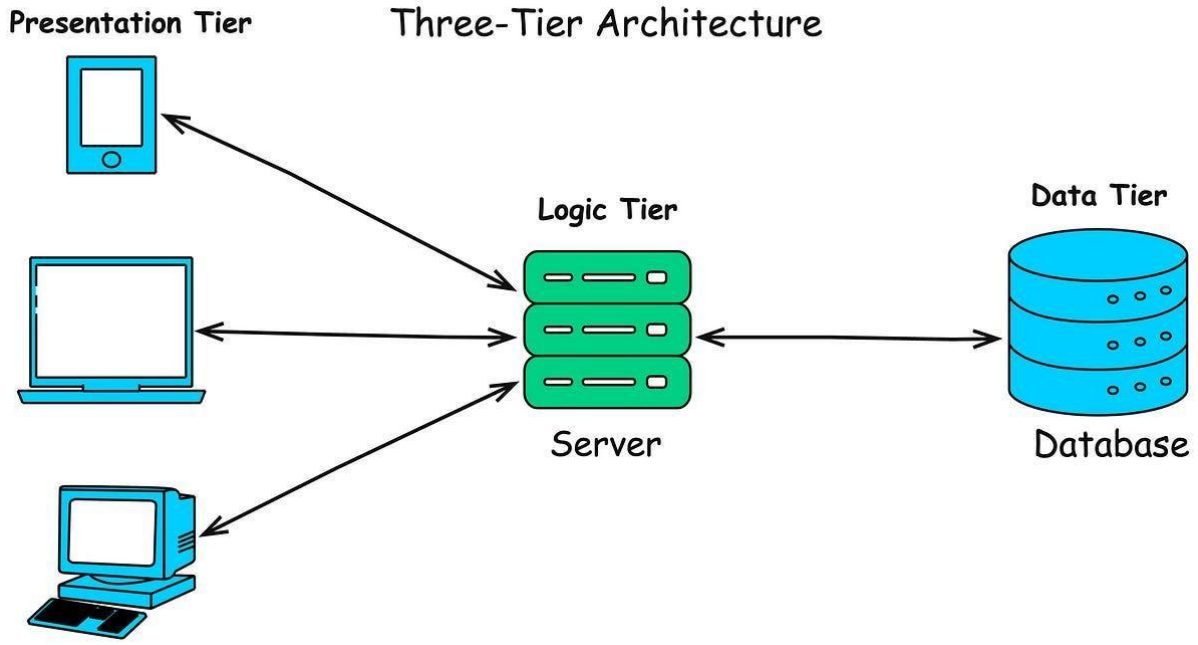
System Design and Architecture

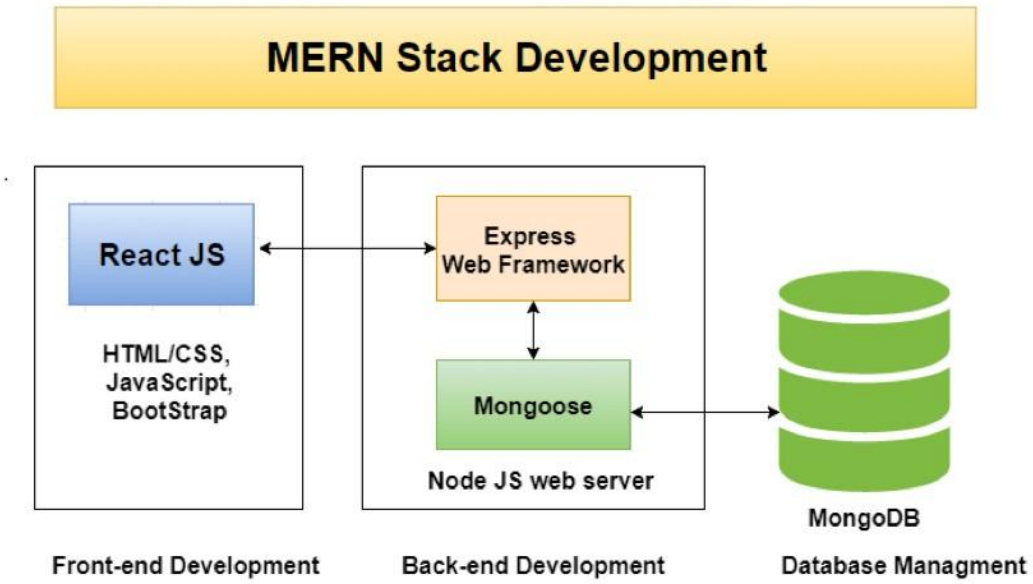
4.1 System Architecture

The Trade Tracker system follows a **Three-Tier Architecture**, which divides the application into three main layers: Client Layer, Application Layer, and Data Layer.
This architecture improves system scalability, maintainability, and performance.

Architecture Components

Client (Frontend)
Application Server (Backend)
Database (MongoDB)





4

Description (paste below diagram):

Client:
The user interacts with the system using a web browser interface built with React.js.

Application Server:
The server processes user requests such as login, add trade, update trade, and generate reports.

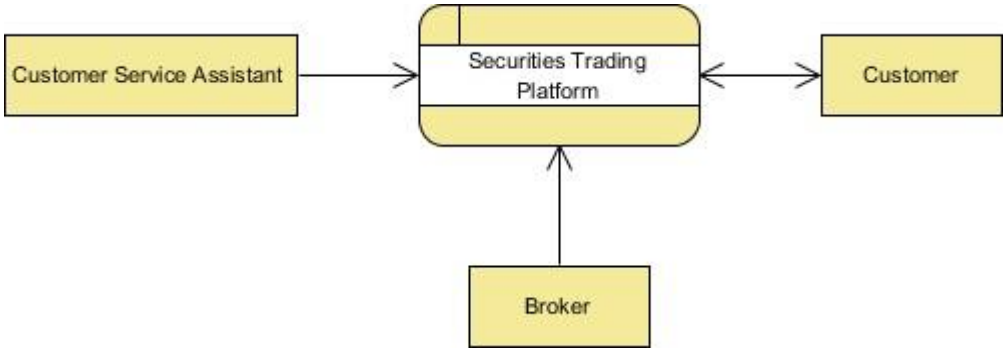
Database:
MongoDB stores user accounts, trade records, and performance data securely.

4.2 Data Flow Diagrams (DFDs)

Data Flow Diagrams illustrate how data moves through the Trade Tracker system.
They show the interaction between users, processes, and the database.

4.2.1 Context-Level Diagram (DFD-0)

The Context-Level Diagram shows the overall interaction between the user and the Trade Tracker system.



4

Description:

External Entity:

User

Main Process: Trade

Tracker System Data

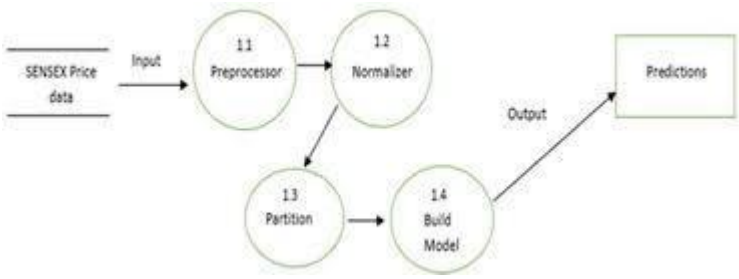
Flows:

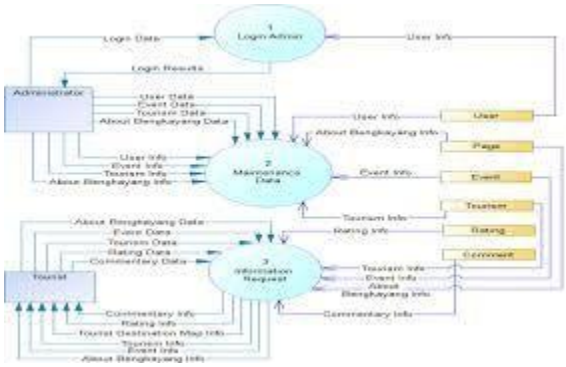
User Registration
Login Request
Trade Entry
Trade History
Performance Report

This diagram shows the overall communication between the user and the system.

4.2.2 Level-1 Diagram (DFD-1)

The Level-1 DFD shows the main processes inside the Trade Tracker system.





4

Description:

Processes:

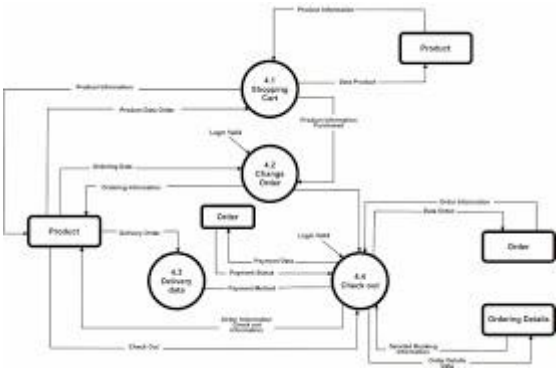
1. User Management
2. Trade Management
3. Analytics & Reports Data Stores:

User Database
Trade Database

This diagram shows how system modules interact with the database.

4.2.3 Level-2 Diagram (Add Trade Process)

The Level-2 DFD provides a detailed view of the trade entry process.



4

Description:

External Entity:

User

Processes:

Enter Trade Details
Validate Trade Data
Store Trade Record
Update Trade Record
Delete Trade Record Data
Store:

Trade Database

This diagram represents the detailed workflow of trade processing.

4.3 Database Design

The database design defines how data is structured and stored in the Trade Tracker system.
MongoDB is used as the primary database to store user and trade information.

4.3.1 Database Schema (E-R Diagram)

The Entity-Relationship Diagram shows the relationship between system entities.

4

Description:

Entities:

User
Trade Analytics

Relationships:

One User can create multiple Trades Each
Trade belongs to one User

This represents a **One-to-Many relationship** between User and Trade.

4.3.2 Data Dictionary

The data dictionary defines the structure of database collections used in the Trade Tracker system.
It describes each field name, data type, constraints, and purpose within the database.

Table 4.1: Users Collection

Field Name — Data Type — Constraints — Description

User_ID — ObjectId — Primary Key — Unique identifier for each user
Name — String — Required — Full name of the user
Email — String — Required, Unique — Email address used for login
Password — String — Required — Encrypted password
Role — String — Required — User role (User / Admin)
Created_At — Date — Default: Current Date — Account creation timestamp

---

Table 4.2: Trades Collection

Field Name — Data Type — Constraints — Description

Trade_ID — ObjectId — Primary Key — Unique identifier for each trade
User_ID — ObjectId — Foreign Key — Reference to the user

Symbol — String — Required — Trading pair or asset name
Direction — String — Required — Buy or Sell
Entry_Price — Number — Required — Price at trade entry
Exit_Price — Number — Optional — Price at trade exit
Stop_Loss — Number — Optional — Stop loss value
Take_Profit — Number — Optional — Take profit value
Profit_Loss — Number — Calculated — Profit or loss result
Trade_Date — Date — Required — Date of the trade

Table 4.3: PsychologyLog Collection

Field Name — Data Type — Description

Log_ID — ObjectId — Unique identifier for psychology record
User_ID — ObjectId — Reference to user
Emotion — String — User emotional state during trade
Confidence_Level — Number — Confidence score
Notes — String — User reflections
Created_Date — Date — Record creation date

Table 4.4: PropFirmRule Collection

Field Name — Data Type — Description

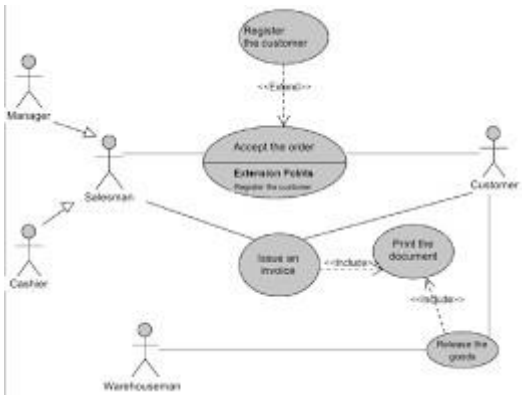
Rule_ID — ObjectId — Unique identifier for rule
User_ID — ObjectId — Reference to user
Firm_Name — String — Name of prop firm
Max_Drawdown — Number — Maximum allowed loss
Profit_Target — Number — Target profit value Current_Status
— String — Rule status

Chapter 5 UML Modeling

UML (Unified Modeling Language) diagrams are used to represent the structure and behavior of the Trade Tracker system.
These diagrams help developers and users understand how the system works and how different components interact with each other.

5.1 Use Case Diagram

The Use Case Diagram illustrates the interaction between users and the main functions of the Trade Tracker system.
It shows what actions the user can perform within the system.



4

Description (paste below diagram) Actors:

User
Administrator **Use**
Cases:

Register
Login
Add Trade
Update Trade
Delete Trade

View Trade History
Generate Reports Logout

This diagram shows how users interact with the system to perform tradingrelated activities.

5.2 Activity Diagrams

Activity diagrams represent the workflow of different processes in the system. They show the sequence of actions performed by the user and the system.

5.2.1 Activity Diagram for User Registration

This diagram shows the step-by-step process of user registration in the Trade Tracker system.

4

Description

Process Steps:

Start
Enter User Details
Validate Input
Store Data in Database
Display Success Message End

This diagram illustrates the workflow for creating a new user account.

5.2.2 Activity Diagram for Add Trade

This diagram shows how a user adds a new trade record into the system.

4

Description

Process Steps:

Start
Enter Trade Details
Validate Trade Data
Save Trade Record
Display Confirmation End

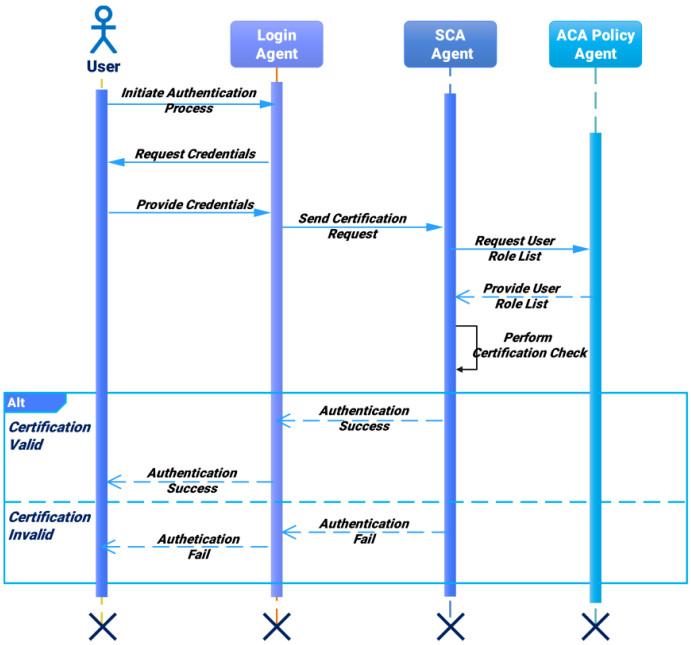
This diagram represents the workflow of recording a trade in the system.

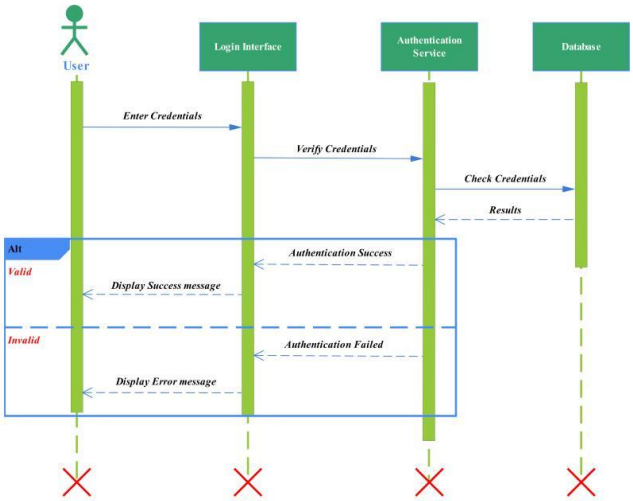
5.3 Sequence Diagrams

Sequence diagrams show how different components of the system interact over time.

5.3.1 Sequence Diagram for User Login

This diagram shows the sequence of actions when a user logs into the system.





4

Description

Actors and Components:

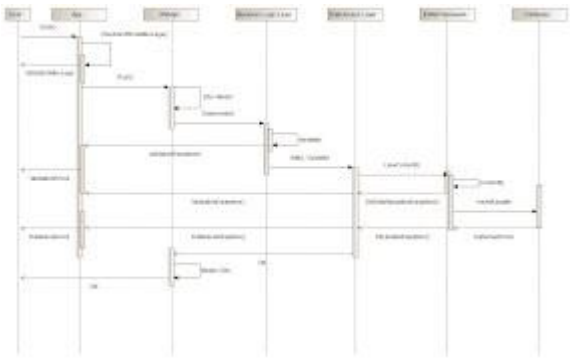
User
Frontend
Backend
Database

Process Flow:

User enters login credentials
Frontend sends request to backend Backend
validates user data
Database returns verification result
System displays dashboard

5.3.2 Sequence Diagram for Add Trade

This diagram shows the interaction between system components when a user records a new trade.



4

Description

Actors and Components:

User
Frontend
Backend
Database

Process Flow:

User enters trade details
Frontend sends trade data to backend
Backend validates data

Chapter 6 Implementation and UI Design

6.1 Implementation Details

The Trade Tracker system is implemented using the **MERN stack**, which includes MongoDB, Express.js, React.js, and Node.js.
The system follows a modular architecture where each component performs a specific function.
This structure improves maintainability, scalability, and performance.

The backend server is responsible for handling user requests, processing data, and communicating with the database.
The frontend provides an interactive interface that allows users to record trades, analyze performance, and manage their accounts.

The system includes secure authentication, trade management features, and performance analytics tools that help users monitor their trading activities effectively.

6.2 Module Breakdown

The Trade Tracker system is divided into several modules to ensure efficient operation and easy management.

Core Modules:

Authentication Module
Handles user registration, login, and logout processes.
Ensures secure access to the system using authentication mechanisms.

Trade Management Module
Allows users to add, update, delete, and view trade records.
Stores trade data in the database and retrieves information when required.

Analytics Module
Generates performance reports and statistics such as profit, loss, and win rate.
Helps users evaluate their trading performance.
Psychology Module
Stores user emotions and reflections related to trading activities.
Helps users analyze behavioral patterns and improve decision-making.

Prop Firm Module
Tracks trading rules such as maximum drawdown and profit targets. Helps users monitor compliance with trading firm requirements.

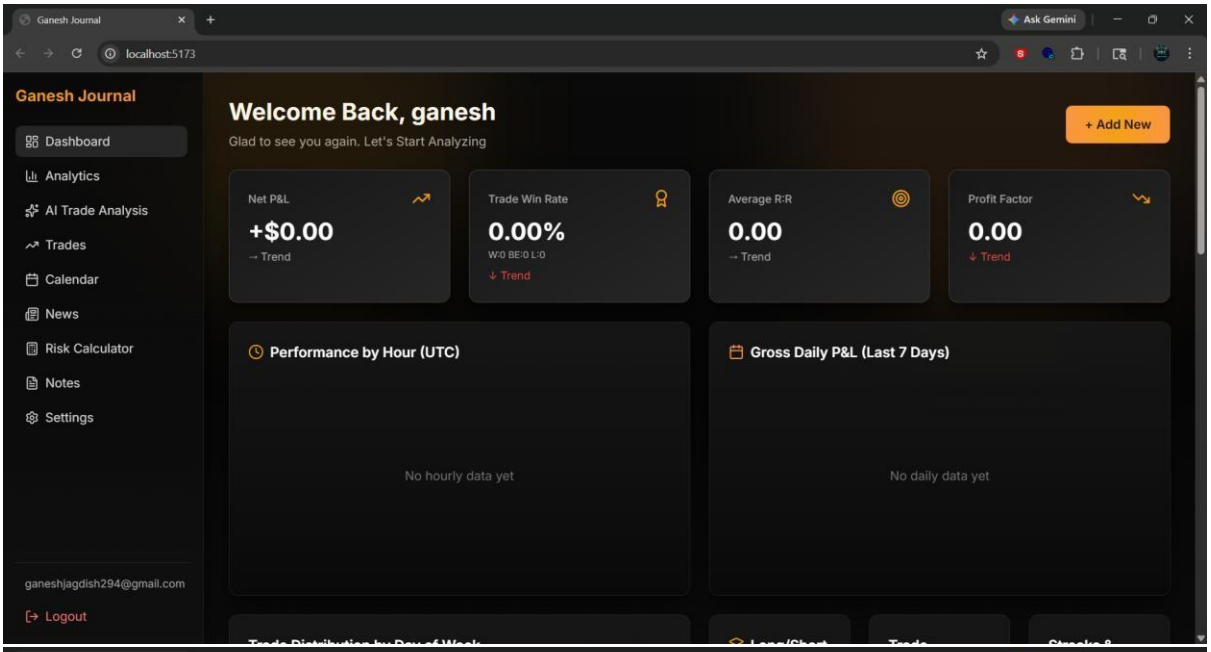
Media Upload Module
Allows users to upload trade screenshots and related files. Stores media data securely in the database.

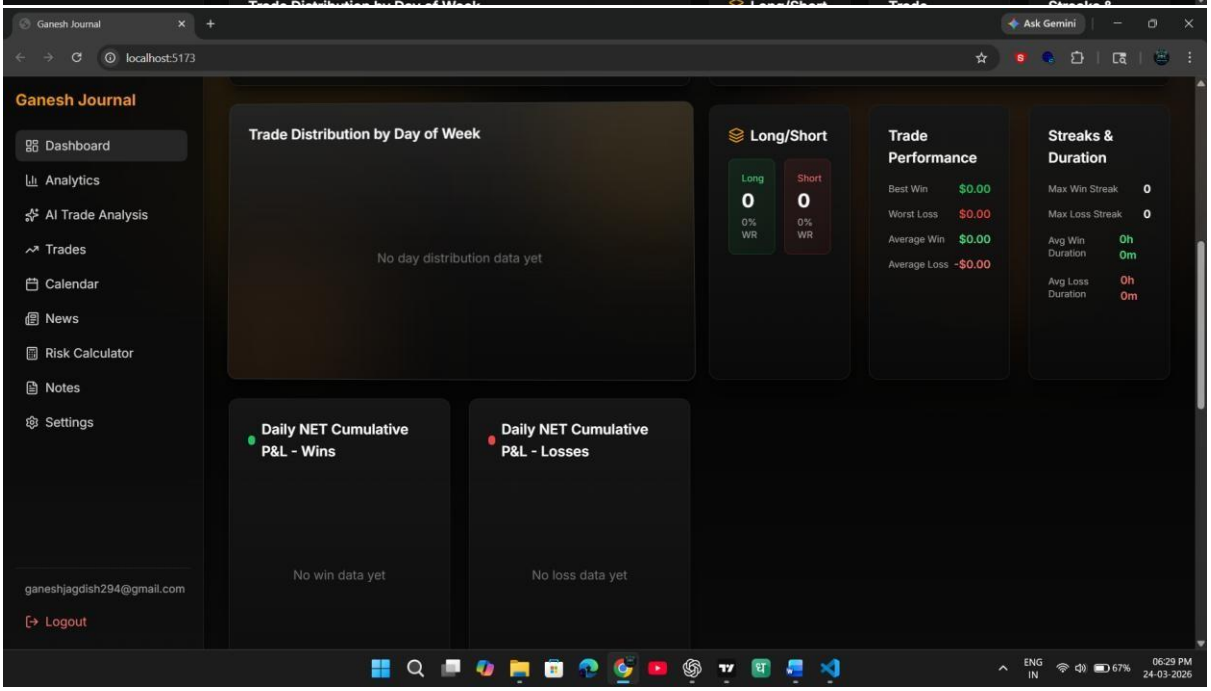
6.3 UI/UX Design (Input/Output Screens)

The user interface of the Trade Tracker system is designed to be simple, responsive, and user-friendly.
It allows users to navigate easily and perform tasks efficiently.

The following figures describe the main user interface screens of the system.

Figure 6.1: UI Screenshot — Dashboard



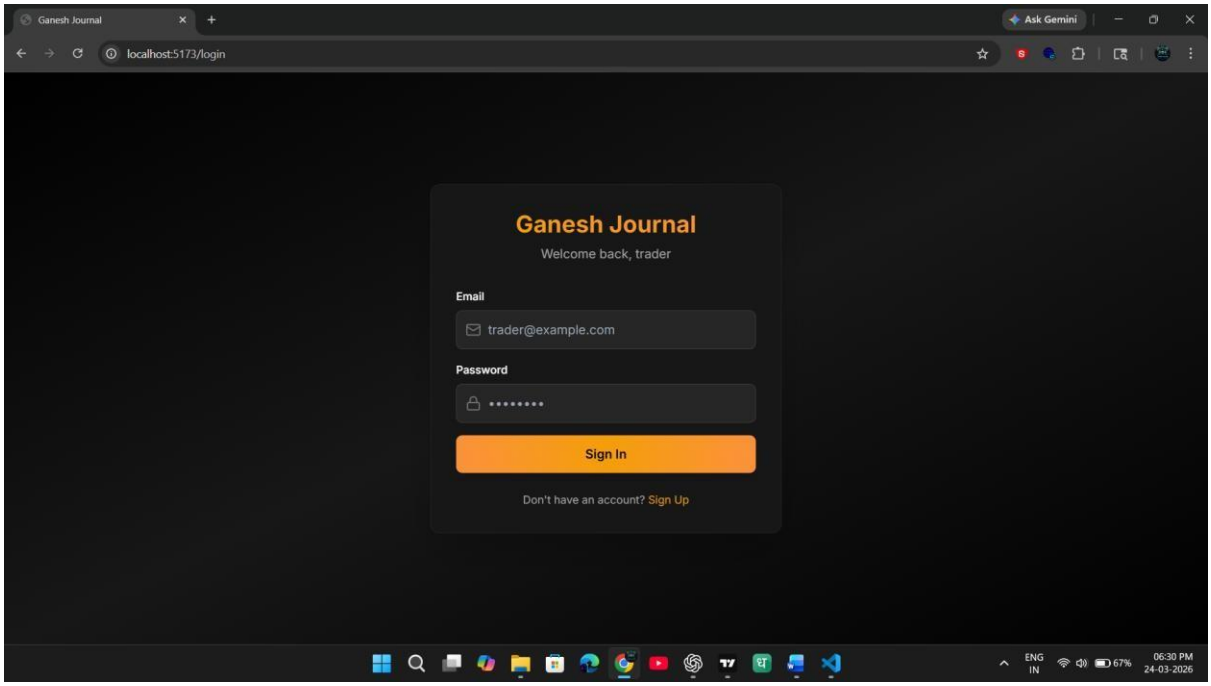


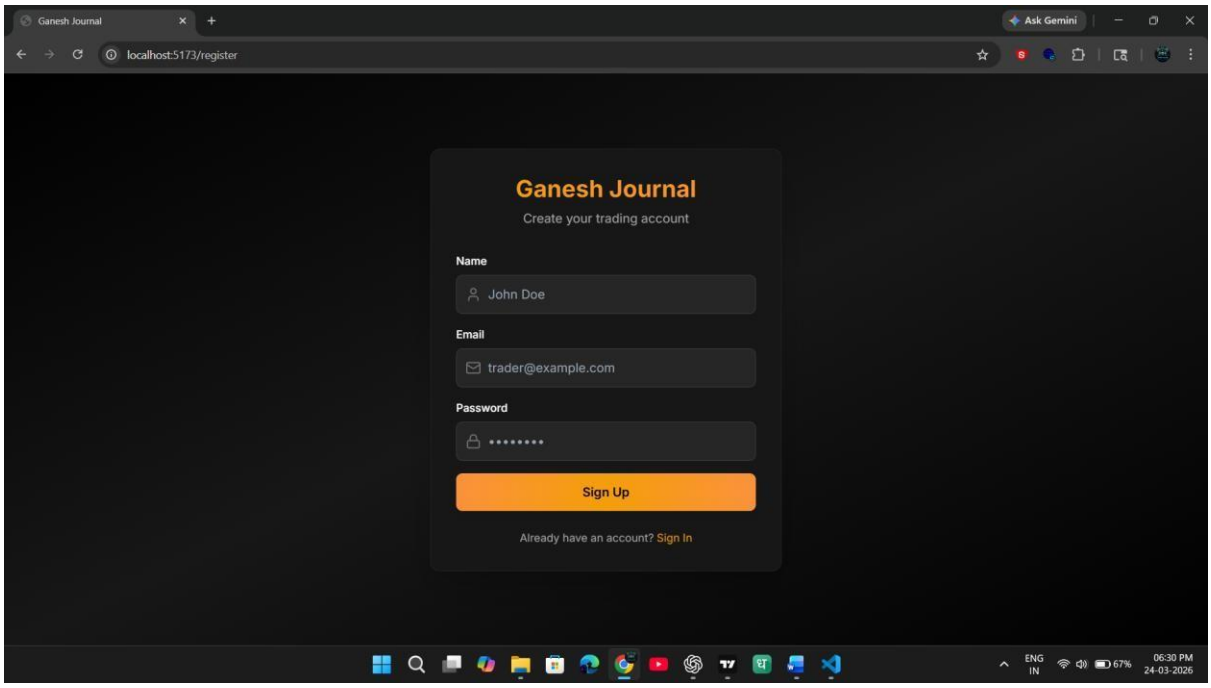
4

Description:

The dashboard is the main screen of the Trade Tracker system.
It displays important information such as total trades, profit and loss, win rate, and recent trading activity.
Users can quickly view their trading performance and access different features from the dashboard.

Figure 6.2: UI Screenshot — Login Page



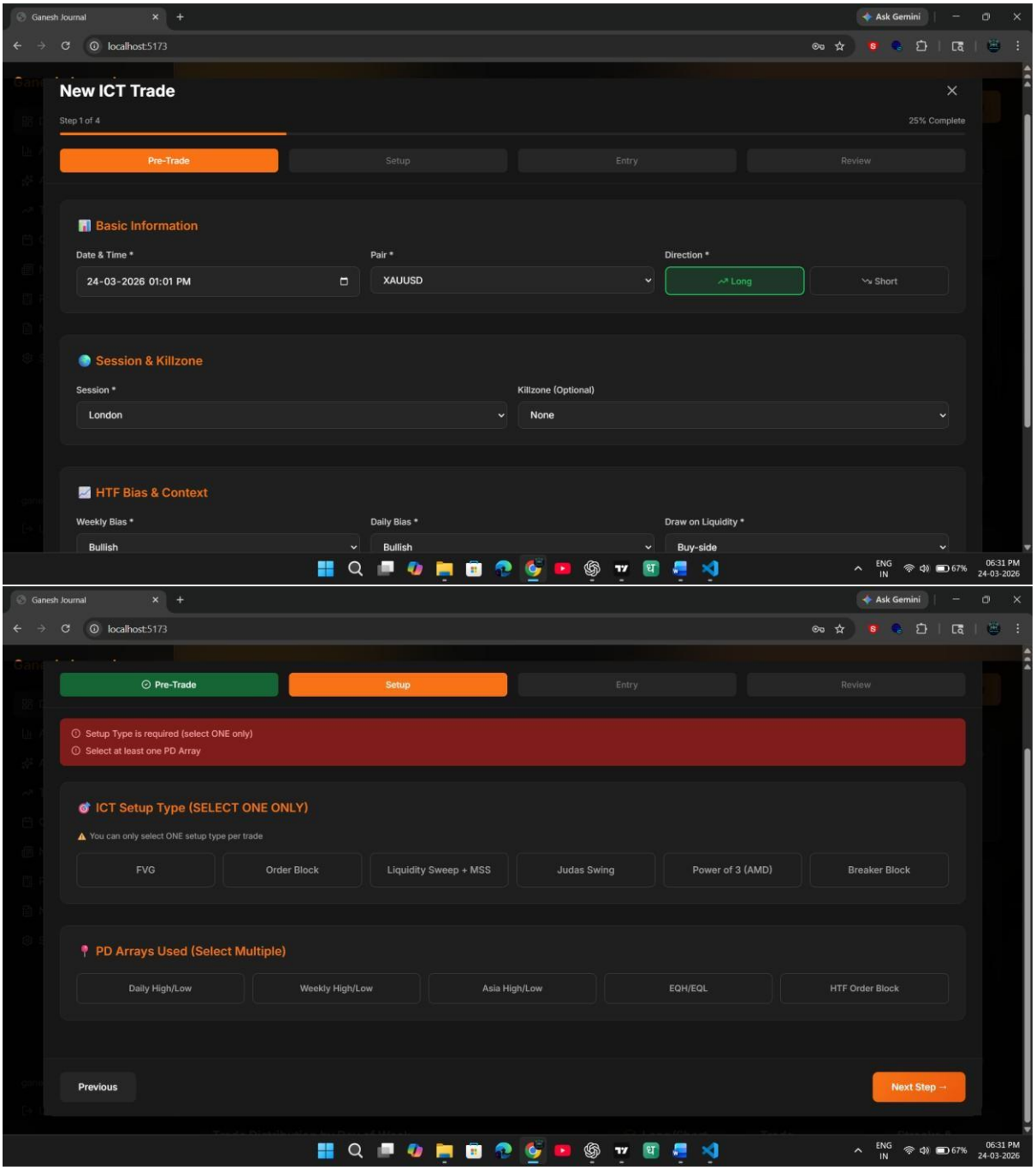


4

Description:

The login page allows users to enter their email and password to access the system.
It includes secure authentication features to protect user data. The interface is simple and easy to use.

Figure 6.3: UI Screenshot — Add Trade Page

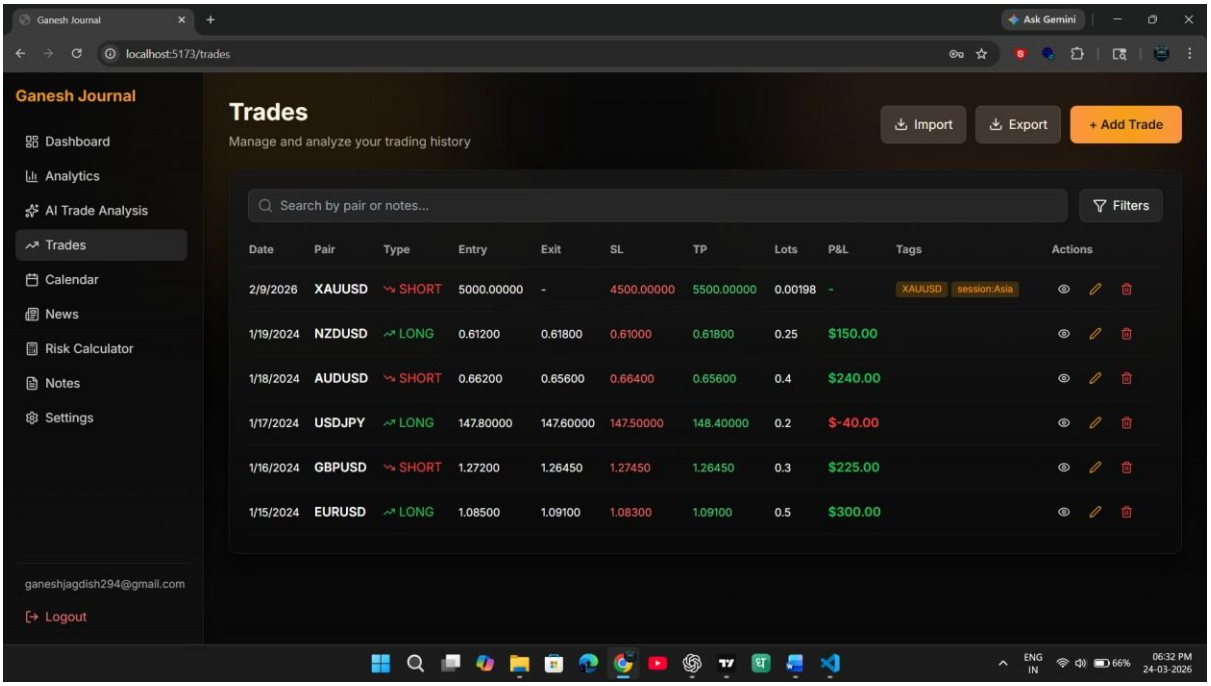


4

Description:

The Add Trade page allows users to enter details such as trading symbol, entry price, exit price, stop loss, and profit or loss.
This screen is used to record new trading transactions.

Figure 6.4: UI Screenshot — Trade History Page



4

Description:

The Trade History page displays all recorded trades in a table format. Users can view, edit, or delete their trade records from this page.

Chapter 7

System Testing

7.1 Testing Strategy

System testing is an essential phase in the software development process that ensures the system functions correctly and meets user requirements. The Trade Tracker system was tested using different testing methods to verify system performance, accuracy, and reliability.

The testing process focused on identifying errors, validating system functionality, and ensuring smooth operation of all modules.
Testing helps improve system quality and ensures that the application works as expected in real-world scenarios.

The following testing methods were used:

Unit Testing
Unit testing involves testing individual components of the system separately. Each module, such as user authentication and trade management, was tested to ensure proper functionality.

Integration Testing
Integration testing verifies the interaction between different modules of the system.
This testing ensures that data flows correctly between the frontend, backend, and database components.

System Testing
System testing evaluates the complete system to ensure that all features work together as expected.
It includes testing user login, trade entry, report generation, and data storage processes.

User Acceptance Testing (UAT)
User acceptance testing ensures that the system meets user expectations and requirements.
Real users tested the system to confirm that it performs correctly and provides a satisfactory user experience.

7.2 Sample Test Cases

The following table presents sample test cases used to verify the functionality of the Trade Tracker system.

Table 7.1: User Login Test Case

Test Case ID — TC-01

Test Case Name — User Login

Objective — Verify that the user can log in successfully

Input — Valid email and password

Expected Result — User dashboard should be displayed

Actual Result — Login successful

Status — Passed

Table 7.2: Add Trade Test Case

Test Case ID — TC-02

Test Case Name — Add Trade

Objective — Verify that the user can add a new trade record

Input — Valid trade details

Expected Result — Trade record should be saved in database

Actual Result — Trade added successfully

Status — Passed

Table 7.3: Update Trade Test Case

Test Case ID — TC-03

Test Case Name — Update Trade

Objective — Verify that the user can update an existing trade
Input — Updated trade details

Expected Result — Trade record should be updated

Actual Result — Trade updated successfully

Status — Passed

Table 7.4: Delete Trade Test Case

Test Case ID — TC-04

Test Case Name — Delete Trade

Objective — Verify that the user can delete a trade record

Input — Selected trade record

Expected Result — Trade record should be removed from database

Actual Result — Trade deleted successfully

Status — Passed

Chapter 8

Conclusion

8.1 Project Summary

The **Trade Tracker** project successfully achieved its primary objective of developing a web-based system for managing and analyzing trading activities. The system provides users with an efficient platform to record trades, monitor performance, and manage trading data in an organized manner.

By using the **MERN stack**, the application ensures fast performance, scalability, and secure data management.
The system includes important features such as user authentication, trade management, performance analytics, and reporting tools.
These features help users improve their trading discipline and make better decisions based on accurate data.

The project demonstrates how modern web technologies can be used to build a reliable and user-friendly application for managing trading information.
Overall, the Trade Tracker system provides a practical solution for traders who want to track their performance and maintain accurate records of their trading activities.

8.2 Limitations

Although the Trade Tracker system provides many useful features, there are certain limitations that can be improved in future versions of the application.

- The system currently depends on manual data entry for recording trades.
- Real-time market data integration is not implemented in the current version.
- The system does not support mobile application functionality.
- Advanced security features such as two-factor authentication are not included.
- Cloud-based backup and synchronization features are limited.

These limitations provide opportunities for further development and enhancement of the system.

8.3 Future Scope

The Trade Tracker system can be expanded and improved by adding new features and technologies in future versions.

Possible future enhancements include:

- Integration with real-time financial market data APIs.
- Development of a mobile application version of the system.
- Implementation of advanced data visualization and analytics tools.
- Addition of automated risk management and trade alerts.
- Implementation of cloud storage and backup systems.
- Integration of artificial intelligence for trading analysis and recommendations.

These improvements will enhance system functionality and provide better support for users in managing their trading activities.

References

1. MongoDB Documentation — https://www.mongodb.com
2. React.js Official Website — https://reactjs.org
3. Node.js Documentation — https://nodejs.org
4. Express.js Documentation — https://expressjs.com
5. MDN Web Docs — https://developer.mozilla.org
6. JavaScript Documentation — https://www.w3schools.com

Bibliography

1. Pressman, Roger S.
   Software Engineering: A Practitioner's Approach McGraw-Hill Education
2. Sommerville, Ian
   Software Engineering Pearson Education
3. Duckett, Jon
   JavaScript and JQuery: Interactive Front-End Web Development Wiley Publications